

Working with Hive on HiveQL

Installation, metastore configuration, troubleshooting and query execution on Ubuntu — a complete step-by-step procedure.

Contents

Part 0	Prerequisites and installation — Java, SSH, MySQL, Hadoop, Hive (Steps 1–10)
Part A	Preparing and starting the Hadoop cluster (Steps 11–18)
Part B	Configuring the Hive metastore on MySQL (Steps 19–25)
Part C	Fixing Hive's scratch directories (Step 26)
Part D	Launching Hive and executing HiveQL (Steps 27–33)
Appendix A	Troubleshooting reference
Appendix B	HiveQL quick reference
Appendix C	Condensed command sequence

Environment Used

Operating System	Ubuntu 24.04 LTS
Hadoop	3.4.0 (HADOOP_HOME=/usr/local/hadoop)
Hive	2.3.9 (HIVE_HOME=/usr/local/hive)
Java	OpenJDK 1.8.0_492 (JDK 8)
Metastore RDBMS	MySQL 8.0.46
HDFS data root	/usr/local/hadoop_tmp
Linux user	cse — replace with your own username throughout

How to read this document: each step gives the command, the output you should expect, and — where a problem is commonly hit — the exact error message and its fix. Part 0 builds the environment from a clean Ubuntu machine; Parts A–D start the cluster, wire Hive to MySQL, and run queries.

Part 0 — Prerequisites and Installation

Skip to Part A if Hadoop and Hive are already installed and `echo $HADOOP_HOME` returns a path.

Step 1. Update the system and install Java 8

```
sudo apt update
sudo apt install openjdk-8-jdk -y
java -version
```

```
openjdk version "1.8.0_492"
```

Version matters. Hive 2.3.9 requires JDK 8. Newer JDKs (11, 17, 21) cause reflection and class-loading failures. If several JDKs are installed, select 8 with `sudo update-alternatives --config java`.

Step 2. Install and configure passwordless SSH

Hadoop's `start-dfs.sh` and `start-yarn.sh` launch daemons over SSH, even on a single machine.

```
sudo apt install ssh pdsh -y

ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 0600 ~/.ssh/authorized_keys

ssh localhost
```

The `ssh localhost` test must succeed without asking for a password. Type `exit` to return.

Step 3. Install MySQL server

```
sudo apt install mysql-server -y
sudo systemctl status mysql
```

MySQL will host the Hive metastore in Part B. Confirm the service shows `active (running)`.

Step 4. Download and extract Hadoop

```
cd ~/Downloads
wget https://archive.apache.org/dist/hadoop/common/hadoop-3.4.0/hadoop-3.4.0.tar.gz
tar -xzvf hadoop-3.4.0.tar.gz
sudo mv hadoop-3.4.0 /usr/local/hadoop
sudo chown -R $USER:$USER /usr/local/hadoop
```

Current releases live on `dldn.apache.org` and older ones move to `archive.apache.org`. If the URL 404s, browse the archive directory and take the matching `.tar.gz`. Use whichever version your lab handout specifies.

Step 5. Download and extract Hive

```
cd ~/Downloads
wget https://archive.apache.org/dist/hive/hive-2.3.9/apache-hive-2.3.9-bin.tar.gz
tar -xzvf apache-hive-2.3.9-bin.tar.gz
sudo mv apache-hive-2.3.9-bin /usr/local/hive
sudo chown -R $USER:$USER /usr/local/hive
```

Step 6. Set environment variables

```
nano ~/.bashrc
```

Append the following block at the end of the file:

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export HIVE_HOME=/usr/local/hive

export PATH=$PATH:$JAVA_HOME/bin
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export PATH=$PATH:$HIVE_HOME/bin

export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
```

Apply and verify:

```
source ~/.bashrc
echo $HADOOP_HOME
echo $HIVE_HOME
```

```
/usr/local/hadoop
/usr/local/hive
```

Step 7. Set `JAVA_HOME` inside `hadoop-env.sh`

```
nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
```

Find the commented `# export JAVA_HOME=` line and replace it with an explicit path:

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
```

Hadoop's start scripts do not reliably inherit `JAVA_HOME` from the shell. Omitting this produces *"JAVA_HOME is not set and could not be found"* at startup.

Step 8. Configure the four Hadoop XML files

All four live in `$HADOOP_HOME/etc/hadoop/`. Place each property block inside the existing `<configuration>` tags.

(a) core-site.xml — defines the HDFS endpoint

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

(b) hdfs-site.xml — replication factor and storage directories

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:/usr/local/hadoop_tmp/hdfs/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:/usr/local/hadoop_tmp/hdfs/datanode</value>
  </property>
</configuration>
```

`dfs.replication` is 1 because there is only one node to replicate to.

(c) mapred-site.xml — run MapReduce under YARN

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

(d) yarn-site.xml — enable the MapReduce shuffle service

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.nodemanager.env-whitelist</name>
    <value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,CLASSPATH_PREPEND_DISTCACHE,HADOOP_YARN_HOME,HADOOP_MAPRED_HOME</value>
  </property>
</configuration>
```

Step 9. Create the HDFS storage directories

```
sudo mkdir -p /usr/local/hadoop_tmp/hdfs/namenode
sudo mkdir -p /usr/local/hadoop_tmp/hdfs/datanode
sudo chown -R $USER:$USER /usr/local/hadoop_tmp
sudo chmod -R 755 /usr/local/hadoop_tmp
```

These must be owned by the user who will run the daemons — ownership problems here are the single most common cause of a `DataNode` that refuses to start.

Step 10. Format HDFS and do a first start

```
hdfs namenode -format
start-dfs.sh
start-yarn.sh
jps
```

```
NameNode
DataNode
SecondaryNameNode
ResourceManager
NodeManager
Jps
```

Once all five daemons appear, create the Hive warehouse directories in HDFS:

```
hdfs dfs -mkdir -p /user/hive/warehouse
hdfs dfs -mkdir -p /tmp
hdfs dfs -chmod g+w /user/hive/warehouse
hdfs dfs -chmod g+w /tmp
hdfs dfs -ls /user/hive
```

Web interfaces for checking cluster health: NameNode at <http://localhost:9870> , ResourceManager at <http://localhost:8088> .

If any daemon is missing from `jps` , do not continue — go to Part A, which diagnoses and repairs exactly this situation.

Part A — Preparing and Starting the Hadoop Cluster

This part covers a cluster that exists but will not start cleanly — the usual state at the beginning of a lab session.

Step 11. Verify the environment

```
jps
echo $HADOOP_HOME
```

```
11663 Jps
/usr/local/hadoop
```

Only `Jps` is listed, so no Hadoop daemons are running yet. `HADOOP_HOME` is correctly exported.

Step 12. Start HDFS and check which daemons came up

```
start-dfs.sh
jps
```

```
Starting namenodes on [localhost]
Starting datanodes
localhost: ERROR: Cannot set priority of datanode process 11987
Starting secondary namenodes [cse-ThinkCentre-M73]
```

```
12145 SecondaryNameNode
12271 Jps
```

Problem: only `SecondaryNameNode` is alive. Both `NameNode` and `DataNode` died on startup. The "Cannot set priority" message is generic — the real reason is always in the log files.

Step 13. Read the daemon logs to find the real cause

```
cd $HADOOP_HOME/logs
ls
tail -50 hadoop-hadoopuser-namenode-cse-ThinkCentre-M73.log
tail -50 hadoop-hadoopuser-datanode-cse-ThinkCentre-M73.log
grep -i "error\|exception\|failed" hadoop-hadoopuser-datanode-cse-ThinkCentre-M73.log | tail -30
```

Two distinct root causes appear in the `DataNode` log:

```
org.apache.hadoop.util.DiskChecker$DiskErrorException:
  Directory is not readable: /usr/local/hadoop_tmp/hdfs/datanode
... Too many failed volumes - current valid volumes: 0, volumes configured: 1

java.io.IOException: Incompatible clusterIDs in /usr/local/hadoop_tmp/hdfs/datanode:
  namenode clusterID = CID-2faadd1-...; datanode clusterID = CID-9fdeb7dc-...
```

- **Cause 1 — permissions.** The data directories were created by a different user (`hadoopuser`), so the current user `cse` cannot read them.
- **Cause 2 — cluster ID mismatch.** The `NameNode` was reformatted at some point but the `DataNode` still carries the old cluster ID, so it refuses to register.

Step 14. Confirm the configured data directories

```
cat $HADOOP_HOME/etc/hadoop/hdfs-site.xml
```

Check that `dfs.namenode.name.dir` and `dfs.datanode.data.dir` match the paths named in the error messages, and that `dfs.replication` is 1 for a single-node setup.

Step 15. Stop all daemons before repairing

```
stop-dfs.sh
stop-yarn.sh
```

Never modify HDFS storage directories while daemons are running.

Step 16. Fix ownership and clear the stale `DataNode` directory

```
sudo mkdir -p /usr/local/hadoop_tmp/hdfs/namenode
sudo mkdir -p /usr/local/hadoop_tmp/hdfs/datanode
sudo chown -R cse:cse /usr/local/hadoop_tmp
sudo chmod -R 755 /usr/local/hadoop_tmp
rm -rf /usr/local/hadoop_tmp/hdfs/datanode/*
```

The `chown` fixes Cause 1; emptying the `datanode` directory fixes Cause 2 by discarding the old cluster ID.

Substitute your own user. `cse:cse` is `user:group`. Run `whoami` if unsure.

Step 17. Format the `NameNode`

```
hdfs namenode -format
```

When prompted, type `Y`:

```
Re-format filesystem in Storage Directory root= /usr/local/hadoop_tmp/hdfs/namenode; location= null ? (Y or N) Y
...
INFO common.Storage: Storage directory /usr/local/hadoop_tmp/hdfs/namenode has been successfully formatted.
```

Destructive operation. Formatting erases all existing HDFS data. It is only appropriate on a lab or development cluster.

Step 18. Start HDFS and YARN, then verify

```
start-dfs.sh
jps
start-yarn.sh
jps
```

```
13296 NameNode
13466 DataNode
13666 SecondaryNameNode
13886 ResourceManager
14023 NodeManager
14358 Jps
```

All five daemons are now running. Do not proceed to Hive until this output matches — Hive stores its warehouse in HDFS and submits jobs to YARN.

Part B — Configuring the Hive Metastore on MySQL

Step 19. First attempt to launch Hive

```
hive
hive> show databases;
```

```
FAILED: SemanticException org.apache.hadoop.hive.ql.metadata.HiveException:
java.lang.RuntimeException: Unable to instantiate
org.apache.hadoop.hive.ql.metadata.SessionHiveMetaStoreClient
```

Exit with `exit`; . This error means Hive cannot reach a usable metastore database.

You may also see repeated `SLF4J: Class path contains multiple SLF4J bindings` warnings. These are harmless — Hive and Hadoop each ship a binding. They can be ignored throughout.

Step 20. Check the current metastore status

```
schematool -dbType mysql -info
```

```
Metastore connection URL: jdbc:derby;;databaseName=metastore_db;create=true
Metastore Connection Driver : org.apache.derby.jdbc.EmbeddedDriver
Metastore connection User: APP
org.apache.hadoop.hive.metastore.HiveMetaException: Failed to get schema version,
Cause:Table/View 'VERSION' does not exist.
*** schemaTool failed ***
```

Two facts confirmed: Hive is still pointing at the default embedded Derby database, and no schema has ever been created.

Why not just use Derby? The embedded Derby metastore allows only one Hive session at a time and stores its database in whichever directory you launched Hive from. MySQL gives a persistent, shareable metastore.

Step 21. Create `hive-site.xml` from the template

```
cd $HIVE_HOME/conf
ls
cp hive-default.xml.template hive-site.xml
ls
```

A fresh Hive installation ships only `.template` files — `hive-site.xml` does not exist until you create it. Confirm it now appears in the listing.

Step 22. Point the four JDO properties at MySQL

Edit `$HIVE_HOME/conf/hive-site.xml` with `nano`, or apply the `sed` commands below. Verify each change with the matching `grep`.

(a) Connection URL

```
sed -i '/<name>javax.jdo.option.ConnectionURL</name>/,/value>/s#<value>.*</value>#<value>jdbc:mysql://localhost:3306/metastore?
createDatabaseIfNotExist=true</value>#' /usr/local/hive/conf/hive-site.xml

grep -A2 "javax.jdo.option.ConnectionURL" /usr/local/hive/conf/hive-site.xml
```

(b) JDBC driver class

```
sed -i 's#org.apache.derby.jdbc.EmbeddedDriver#com.mysql.cj.jdbc.Driver#g' /usr/local/hive/conf/hive-site.xml

grep -A2 "ConnectionDriverName" /usr/local/hive/conf/hive-site.xml
```

(c) Metastore username

```
sed -i 's#<value>APP</value>#<value>hiveuser</value>#' /usr/local/hive/conf/hive-site.xml

grep -A2 "ConnectionUserName" /usr/local/hive/conf/hive-site.xml
```

(d) Metastore password — locate the line number, then edit it in `nano`:

```
grep -n -A2 "javax.jdo.option.ConnectionPassword" /usr/local/hive/conf/hive-site.xml | head -5
nano /usr/local/hive/conf/hive-site.xml      # set <value>mine</value>
sed -n '525,535p' /usr/local/hive/conf/hive-site.xml
```

All four values should now read:

```
javax.jdo.option.ConnectionURL      jdbc:mysql://localhost:3306/metastore?createDatabaseIfNotExist=true
javax.jdo.option.ConnectionDriverName com.mysql.cj.jdbc.Driver
javax.jdo.option.ConnectionUserName  hiveuser
javax.jdo.option.ConnectionPassword mine
```

Paste carefully. A pasted command may produce `[200~sed: command not found`. Those `[200~` characters are bracketed-paste escape codes from the terminal. If you see them, retype the command by hand or paste into `nano` instead.

There is a second, unrelated match for `ConnectionPassword` further down the file (inside `hive.conf.hidden.list`, around line 5915). Leave that one alone — only edit the property at roughly line 530.

Step 23. Install the MySQL JDBC driver

```
ls $HIVE_HOME/lib | grep mysql
```

```
mysql-metadata-storage-0.9.2.jar
```

That jar is not a JDBC driver. Download the real connector:

```
cd $HIVE_HOME/lib
sudo wget https://repo1.maven.org/maven2/com/mysql/mysql-connector-j/8.0.33/mysql-connector-j-8.0.33.jar
ls $HIVE_HOME/lib | grep mysql
```

```
mysql-connector-j-8.0.33.jar
mysql-metadata-storage-0.9.2.jar
```

Skipping this step gives: HiveMetaException: Failed to load driver — java.lang.ClassNotFoundException: com.mysql.cj.jdbc.Driver

Step 24. Create the MySQL user, database and privileges

```
sudo mysql
```

```
mysql> SELECT user,host FROM mysql.user;
mysql> CREATE USER 'hiveuser'@'localhost' IDENTIFIED BY 'mine';
mysql> CREATE DATABASE metastore;
mysql> GRANT ALL PRIVILEGES ON metastore.* TO 'hiveuser'@'localhost';
mysql> FLUSH PRIVILEGES;
mysql> SELECT user,host FROM mysql.user;
mysql> exit;
```

The final `SELECT` should now list `hiveuser | localhost` alongside the built-in accounts.

Skipping this step gives: java.sql.SQLException: Access denied for user 'hiveuser'@'localhost' (using password: YES) , SQL error code 1045.

The password `mine` must match exactly what you put in `hive-site.xml` at step 22(d).

Step 25. Initialise the metastore schema

```
schematool -dbType mysql -initSchema
```

```
Metastore connection URL: jdbc:mysql://localhost:3306/metastore?createDatabaseIfNotExist=true
Metastore Connection Driver : com.mysql.cj.jdbc.Driver
Metastore connection User: hiveuser
Starting metastore schema initialization to 2.3.0
Initialization script hive-schema-2.3.0.mysql.sql
Initialization script completed
schemaTool completed
```

The metastore is now built. This command is run once only — running it again on an initialised database will fail.

Part C — Fixing Hive's Scratch Directories

Step 26. Replace the unresolved `${system:...}` variables

Launching `hive` at this point still fails:

```
Exception in thread "main" java.lang.IllegalArgumentException: java.net.URISyntaxException:
Relative path in absolute URI: ${system:java.io.tmpdir%7D/%7Bsystem:user.name%7D
at org.apache.hadoop.hive.ql.session.SessionState.createSessionDirs(SessionState.java:663)
```

Cause: `hive-default.xml.template` contains placeholders such as `${system:java.io.tmpdir}/${system:user.name}` that Hive does not expand when read from `hive-site.xml`. Replace them with absolute paths.

```
grep -n 'system:java.io.tmpdir' $HIVE_HOME/conf/hive-site.xml
nano $HIVE_HOME/conf/hive-site.xml
```

Set these four properties to literal paths:

Property	Value
<code>hive.exec.local.scratchdir</code>	<code>/tmp/hive</code>
<code>hive.downloaded.resources.dir</code>	<code>/tmp/hive_resources</code>
<code>hive.querylog.location</code>	<code>/tmp/hive_query_logs</code>
<code>hive.server2.logging.operation.log.location</code>	<code>/tmp/hive/operation_logs</code>

Then create the directories and open their permissions:

```
mkdir -p /tmp/hive
mkdir -p /tmp/hive_resources
mkdir -p /tmp/hive_query_logs
chmod -R 777 /tmp/hive
chmod -R 777 /tmp/hive_resources
chmod -R 777 /tmp/hive_query_logs
```

Part D — Launching Hive and Executing HiveQL

Step 27. Start the Hive CLI

```
hive
hive> show databases;

OK
default
Time taken: 4.245 seconds, Fetched: 1 row(s)
```

Hive is now fully operational. The `Hive-on-MR is deprecated` warning is informational only.

Step 28. Create a database and table (DDL)

```
hive> CREATE DATABASE studentdb;
hive> SHOW DATABASES;
hive> USE studentdb;

hive> CREATE TABLE student(
  > rollno INT,
  > name STRING,
  > branch STRING,
  > marks INT
  > )
  > ROW FORMAT DELIMITED
  > FIELDS TERMINATED BY ',';

hive> SHOW TABLES;

OK
default
studentdb
...
```

`ROW FORMAT DELIMITED FIELDS TERMINATED BY ','` tells Hive that the underlying data files are comma-separated. Confirm the warehouse directory was created in HDFS:

```
hdfs dfs -ls /user/hive/warehouse

drwxr-xr-x - cse supergroup 0 /user/hive/warehouse/studentdb.db
```

Step 29. First INSERT attempt — and its failure

```
hive> INSERT INTO student VALUES
  > (1,'Ravi','CSE',85),
  > (2,'Priya','CSE',90),
  > (3,'Anil','ISE',78);
```

```
Starting Job = job_1785997552466_0001
Hadoop job information for Stage-1: number of mappers: 0; number of reducers: 0
Ended Job = job_1785997552466_0001 with errors
FAILED: Execution Error, return code 2 from org.apache.hadoop.hive.ql.exec.mr.MapRedTask
```

`SELECT * FROM student;` returns OK with no rows, confirming nothing was written.

Step 30. Diagnose the MapReduce failure

```
yarn logs -applicationId application_1785997552466_0003
cat $HADOOP_HOME/etc/hadoop/mapred-site.xml
tail -50 $(ls -t $HADOOP_HOME/logs/*nodemanager*.log | head -1)
```

`yarn logs` reports no aggregated logs (aggregation is off by default), and `mapred-site.xml` is correct with `mapreduce.framework.name=yarn`. The NodeManager log holds the answer:

```
Container container_1785997552466_0003_02_000001 transitioned from SCHEDULED to RUNNING
WARN DefaultContainerExecutor: Exit code from container ... is : 1
ExitCodeException exitCode=1:
WARN ContainerLaunch: Container launch failed : Container exited with a non-zero exit code 1.
Container ... transitioned from RUNNING to EXITED_WITH_FAILURE
```

Diagnosis: every YARN container dies immediately at launch (exit code 1), before any mapper starts — which is why the job reports 0 mappers and 0 reducers. On a small single-node machine this is typically insufficient container memory or a container-launch environment problem, not a fault in the query.

Shell tip: `tail -50 $HADOOP_HOME/logs/*nodemanager*.log` fails with "*option used in invalid context*" when the wildcard matches several files. The `$(ls -t ... | head -1)` form used above picks the newest log and works reliably.

Step 31. Fix — run the query in Hive local mode

Rather than debugging YARN's container memory, bypass YARN for these small jobs. Restart the CLI and set:

```
hive
hive> set hive.execution.engine;
hive> set hive.exec.mode.local.auto=true;
hive> set hive.exec.local.scratchdir=/tmp/hive;
hive> USE studentdb;
```

```
hive> INSERT INTO TABLE student VALUES
> (1,'Ravi','CSE',85),
> (2,'Priya','CSE',90),
> (3,'Anil','ISE',78);
```

```
Automatically selecting local only mode for query
Launching Job 1 out of 3
Job running in-process (local Hadoop)
2026-08-06 12:50:30,568 Stage-1 map = 100%, reduce = 0%
Ended Job = job_local235257530_0001
Loading data to table studentdb.student
Stage-Stage-1: HDFS Read: 43 HDFS Write: 159 SUCCESS
OK
Time taken: 3.992 seconds
```

Do not forget `USE studentdb`; — these settings apply per session. Running the INSERT before selecting the database returns `SemanticException ... InvalidTableException: Table not found student`.

To make local mode permanent, add `hive.exec.mode.local.auto=true` as a property in `hive-site.xml` instead of typing it each session.

Step 32. Verify the data

```
hive> SELECT * FROM student;
```

```
OK
1  Ravi   CSE   85
2  Priya  CSE   90
3  Anil   ISE   78
Time taken: 0.163 seconds, Fetched: 3 row(s)
```

Failed inserts leave behind temporary tables named `values__tmp__table__1`, `values__tmp__table__2`, etc. They are harmless; remove them with `DROP TABLE values__tmp__table__1;` to keep `SHOW TABLES;` clean.

Step 33. Shut down cleanly

```
hive> exit;

stop-yarn.sh
stop-dfs.sh
jps
```

Always stop YARN before HDFS. `jps` should list only `Jps` afterwards.

Appendix A — Troubleshooting Reference

Error message	Cause	Fix
JAVA_HOME is not set and could not be found	<code>hadoop-env.sh</code> not edited	Set <code>JAVA_HOME</code> explicitly inside <code>hadoop-env.sh</code>
localhost: Permission denied (publickey,password)	Passwordless SSH not configured	<code>ssh-keygen</code> , then append the public key to <code>authorized_keys</code>
ERROR: Cannot set priority of datanode process	Generic wrapper message; real cause is in the DataNode log	Read <code>\$HADOOP_HOME/logs/*datanode*.log</code>
DiskChecker\$DiskErrorException: Directory is not readable	Data directory owned by another user	<code>sudo chown -R user:user /usr/local/hadoop_tmp</code>
Incompatible clusterIDs	NameNode reformatted, DataNode still holds the old cluster ID	Empty the datanode dir, then <code>hdfs namenode -format</code>
Unable to instantiate SessionHiveMetaStoreClient	Metastore schema not initialised	Configure MySQL and run <code>schematool -dbType mysql -initSchema</code>
Table/View 'VERSION' does not exist	Empty metastore database	Run <code>schematool ... -initSchema</code>
No such file or directory: hive-site.xml	Only templates ship with Hive	<code>cp hive-default.xml.template hive-site.xml</code>
ClassNotFoundException: com.mysql.cj.jdbc.Driver	MySQL JDBC connector missing	Download <code>mysql-connector-j-8.0.33.jar</code> into <code>\$HIVE_HOME/lib</code>
Access denied for user 'hiveuser'@'localhost' (1045)	MySQL account absent or password mismatch	<code>CREATE USER</code> + <code>GRANT</code> + <code>FLUSH PRIVILEGES</code>
URISyntaxException: Relative path in absolute URI: \${system:java.io.tmpdir}	Unexpanded placeholders inherited from the template	Replace <code>scratch-dir</code> properties with absolute <code>/tmp/...</code> paths
Return code 2 from MapRedTask; 0 mappers, 0 reducers	YARN containers exit with code 1 at launch	<code>set hive.exec.mode.local.auto=true;</code>
InvalidTableException: Table not found student	No database selected in this session	<code>USE studentdb;</code> before the query
tail: option used in invalid context -- 5	Wildcard matched multiple log files	<code>tail -50 \$(ls -t <pattern> head -1)</code>
[200~sed: command not found	Bracketed-paste escape characters pasted into the shell	Retype the command, or paste inside <code>nano</code>

Appendix B — HiveQL Quick Reference

Category	Statement
Databases	<code>CREATE DATABASE name;</code> <code>SHOW DATABASES;</code> <code>USE name;</code> <code>DROP DATABASE name;</code>
Tables	<code>CREATE TABLE t (col TYPE, ...) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';</code>
Inspect	<code>SHOW TABLES;</code> <code>DESCRIBE t;</code> <code>DESCRIBE FORMATTED t;</code>
Insert rows	<code>INSERT INTO TABLE t VALUES (...), (...);</code>
Bulk load	<code>LOAD DATA LOCAL INPATH '/path/file.csv' INTO TABLE t;</code>
Query	<code>SELECT * FROM t;</code> <code>SELECT branch, AVG(marks) FROM t GROUP BY branch;</code>
Filter / sort	<code>SELECT * FROM t WHERE marks > 80 ORDER BY marks DESC;</code>
Alter / drop	<code>ALTER TABLE t RENAME TO t2;</code> <code>DROP TABLE t;</code>
Session settings	<code>set hive.exec.mode.local.auto=true;</code> <code>set hive.execution.engine;</code>
Data types	<code>INT</code> , <code>BIGINT</code> , <code>FLOAT</code> , <code>DOUBLE</code> , <code>STRING</code> , <code>BOOLEAN</code> , <code>DATE</code> , <code>TIMESTAMP</code>

Note on INSERT. Hive is designed for bulk analytics, not row-level transactions. Each `INSERT ... VALUES` launches a MapReduce job, which is why it is slow. In practice, `LOAD DATA` from a file is the normal way to populate a table.

Appendix C — Condensed Command Sequence

```
# --- Install (once) ---
sudo apt update
sudo apt install openjdk-8-jdk ssh pdsh mysql-server -y
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys

# extract Hadoop to /usr/local/hadoop and Hive to /usr/local/hive
# edit ~/.bashrc : JAVA_HOME, HADOOP_HOME, HIVE_HOME, PATH
# edit hadoop-env.sh : JAVA_HOME
# edit core-site.xml, hdfs-site.xml, mapred-site.xml, yarn-site.xml

# --- Hadoop start / repair ---
stop-dfs.sh; stop-yarn.sh
sudo mkdir -p /usr/local/hadoop_tmp/hdfs/namenode
sudo mkdir -p /usr/local/hadoop_tmp/hdfs/datanode
sudo chown -R $USER:$USER /usr/local/hadoop_tmp
sudo chmod -R 755 /usr/local/hadoop_tmp
rm -rf /usr/local/hadoop_tmp/hdfs/datanode/*
hdfs namenode -format
start-dfs.sh
start-yarn.sh
jps

hdfs dfs -mkdir -p /user/hive/warehouse
hdfs dfs -chmod g+w /user/hive/warehouse

# --- Hive metastore on MySQL ---
cd $HIVE_HOME/conf
cp hive-default.xml.template hive-site.xml
# edit: ConnectionURL / ConnectionDriverName / ConnectionUserName / ConnectionPassword
# edit: hive.exec.local.scratchdir, hive.downloaded.resources.dir,
#       hive.querylog.location, hive.server2.logging.operation.log.location

cd $HIVE_HOME/lib
sudo wget https://repo1.maven.org/maven2/com/mysql/mysql-connector-j/8.0.33/mysql-connector-j-8.0.33.jar

sudo mysql -e "CREATE USER 'hiveuser'@'localhost' IDENTIFIED BY 'mine';
               CREATE DATABASE metastore;
               GRANT ALL PRIVILEGES ON metastore.* TO 'hiveuser'@'localhost';
               FLUSH PRIVILEGES;"

mkdir -p /tmp/hive /tmp/hive_resources /tmp/hive_query_logs
chmod -R 777 /tmp/hive /tmp/hive_resources /tmp/hive_query_logs

schematool -dbType mysql -initSchema

# --- Execute ---
hive
```

```
set hive.exec.mode.local.auto=true;
CREATE DATABASE studentdb;
USE studentdb;
CREATE TABLE student(rollno INT, name STRING, branch STRING, marks INT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
INSERT INTO TABLE student VALUES (1,'Ravi','CSE',85),(2,'Priya','CSE',90),(3,'Anil','ISE',78);
SELECT * FROM student;
exit;
```

Module 4 — Working with Hive on HiveQL. Parts A–D are transcribed from a recorded Ubuntu terminal session; Part 0 reconstructs the installation that preceded it. Paths, usernames and version numbers reflect that specific machine — adjust them to match your own installation and lab handout.